

编译原理·导论笔记

Introduction to Compilation Principle ——详细学习笔记

基于课件 0.Introduction.pdf (中山大学·赵帅) 整理

本笔记说明

本笔记完整覆盖导论课件 34 页内容，分两大板块：**课程行政信息**（第 1–11 页）与**编译原理核心概念**（第 12–34 页）。每个知识点后用 [p 页码] 标注来源幻灯片。所有专业术语首次出现时给出中文 (English) 对照，并配通俗解释。后半部分各阶段的”知识结构图”是整门课的术语全景，本笔记逐一拆解，作为后续章节的”地图”。

目录

1 课程概览与行政信息（第 1–11 页）	1
1.1 课程定位	1
1.2 理论课与实验课	1
1.3 教材与参考资料 [p8]	2
1.4 时间、地点与进度	2
1.5 成绩构成 [p11]	2
2 什么是编译（第 12–18 页）	2
2.1 编译要解决的根本矛盾 [p12]	2
2.2 三种语言执行方式：编译、解释、JIT	3
2.3 计算机系统的层次结构 [p13]	3
2.4 C 语言的完整编译流程 [p16,17]	4
2.5 为什么要学编译原理 [p18]	4
3 编译器的总体结构（第 19–20 页）	4
4 贯穿全课的运行示例	6
5 阶段一：词法分析（第 21–22 页）	6
5.1 词法分析的知识地图 [p22]	6
6 阶段二：语法分析（第 23–25 页）	7
6.1 基础概念 [p24]	7
6.2 两大类分析方法 [p25]	8

1 课程概览与行政信息 (第 1–11 页)	2
7 阶段三：语义分析 (第 26–27 页)	9
7.1 语义分析的知识地图 [p27]	9
8 阶段四：中间代码生成 (第 28–29 页)	10
8.1 中间代码的知识地图 [p29]	10
9 阶段五：代码优化 (第 30–31 页)	11
9.1 代码优化的知识地图 [p31]	11
10 阶段六：目标代码生成 (第 32–33 页)	12
10.1 目标代码生成的知识地图 [p33]	12
11 全流程一图速记	13

1 课程概览与行政信息 (第 1–11 页)

1.1 课程定位

- 课程名称：编译原理 (Introduction to Compilation Principle)，授课教师赵帅，中山大学计算机学院。 [p1,3]
- 适用年级：大三第二学期。 [p2]
- 先修课程：计算机组成原理、汇编语言、数据结构、编程语言设计等。 [p2]
- 贯穿全课的核心问题：一个用高级语言写的程序，最后究竟如何在计算机上跑起来？ [p2]

为什么先修这些课

编译器是”软件”与”硬件”之间的翻译官，所以既要懂高层的语言/数据结构（怎么描述程序），又要懂底层的组成原理/汇编（机器怎么执行）。这些先修课正好覆盖了这两端。

1.2 理论课与实验课

- 理论课：讲解编译核心理论与方法——词法分析、语法分析、语义分析、代码生成、代码优化。 [p2]
- 实验课：独立课程，与理论课协同；用 Java 做语言处理、逆向工程工具等。 [p2]

1.3 教材与参考资料 [p8]

- 主教材：《Compilers: Principles, Techniques, and Tools》（俗称”龙书”，The Dragon Book）第二版，覆盖第 1–6 章 + 附录 A。
- 参考书：《Modern Compiler Implementation in Java》（”虎书”）、《Advanced Compiler Design and Implementation》（”鲸书”）、《程序员的自我修养——链接、装载与库》。

1.4 时间、地点与进度

- 理论课：周一（第 1–9 周）+ 周三（第 1–17 周），第 5–6 节（14:20–16:00），东 D302。[p9]
- 实验课：周三（第 1–17 周），第 7–8 节（16:30–18:10），北实验楼 B202。[p9]

表 1: 学时分配（按编译阶段，单位：周/讲次） [p10]

阶段	课时数
词法分析 (Lexical Analysis)	4
语法分析 (Syntax Analysis)	8 (最重)
语义分析 (Semantic Analysis)	4
中间代码 (Intermediate Code)	3
代码优化 (Code Optimization)	3
目标代码生成 (Code Generation)	2

1.5 成绩构成 [p11]

- 理论课（3 学分 / 54 学时）：平时 40% + 闭卷考试 60%。
- 实验课（1 学分 / 36 学时）：四个项目依次占 10% / 25% / 30% / 35%（所得税计算器 → 改进语言处理程序 → 表达式计算器 → 程序逆向工程工具）。

2 什么是编译 (第 12–18 页)

2.1 编译要解决的根本矛盾 [p12]

- 程序员用高级语言（C/C++/Java/Python/Ruby 等）写程序，关注功能，不关心底层细节。
- 但计算机只懂机器语言（二进制码），无法直接执行高级语言。
- 因此需要一个翻译器，把高级语言翻成机器码，并兼顾查错、性能、可移植性。

定义：编译 (Compilation)

把一种语言（通常是高级源语言 source language）书写的程序，翻译成另一种语言（通常是目标语言 target language，如机器码/汇编）的等价程序的过程。完成这项工作的程序就叫编译器 (Compiler)。

2.2 三种语言执行方式：编译、解释、JIT

编译 (Compile / AOT, Ahead-of-Time) [p13,14]

执行前一次性全文翻译成机器码，再运行。代表语言：C、C++。
特点： 生成的目标程序独立于源程序（改代码要重新编译）； 能分析程序上下文，便于做整体优

化； **性能最好** (所以核心代码常用 C/C++)。

解释 (Interpret) [p13,14]

把源程序作为输入，**边翻译边执行**，逐句进行。代表语言：Python、Ruby。

特点： **不生成独立目标程序**，**可移植性高**；逐句执行，**难以整体优化**； **性能通常一般**。

即时编译 (JIT, Just-In-Time) [p13,15]

运行时才把代码 (或字节码) 编译成机器码，并**缓存**起来供下次复用。代表语言：Java (先编译为字节码 .class，运行时由 JIT 转机器码)。

JIT vs. AOT: JIT 兼具**解释器的灵活性** (有 JIT 即可运行)，性能基本与 AOT 相当；虽有运行时编译的开销，但能利用**运行时特征做动态优化**。

课堂 Quiz: Java 是解释型还是编译型? [p14]

答案：**两者皆是 (混合型)**。Java 源码先被编译成与平台无关的**字节码 (bytecode, .class)**，运行时再由 JVM 通过**解释 + JIT 编译**执行。所以它既不是纯编译，也不是纯解释。

2.3 计算机系统的层次结构 [p13]

理解编译器的位置，要先看”软件如何一层层落到硬件”：

应用程序 Application	软件 <i>SOFTWARE</i>
编译器 / 库 Compiler / Libraries	
操作系统 Operating System	体系结构 <i>ARCHITECTURE</i>
指令集 ISA (Instruction Set)	
计算机组成 System Organization	
电路 Circuits (硬件功能)	硬件 <i>HARDWARE</i>
半导体物理 Semiconductor Physics	

要点：编译器处在**应用程序之下、操作系统之上**，是把”人写的程序”向”机器能跑的指令”转化的关键一层。**ISA (指令集架构)**是软硬件的分界线——上层软件最终都要落到 ISA 定义的指令上。

表 2: C 编译四阶段：每步的输入 → 输出

阶段	做什么	文件变化
预处理 Preprocessing	合并头文件、展开宏定义	.c → .i (仍是 C 代码)
编译 Compiling	把展开后的代码翻成汇编	.i → .s (汇编 assembly)
汇编 Assembling	汇编翻成机器码	.s → .o (可重定位目标文件)
链接 Linking	链接库代码，拼成可执行文件	.o → a.out (机器码可执行)

2.4 C 语言的完整编译流程 [p16,17]

以 `gcc source.c -o a.out` 为例，源程序到可执行文件经过**四步**：

术语小贴士

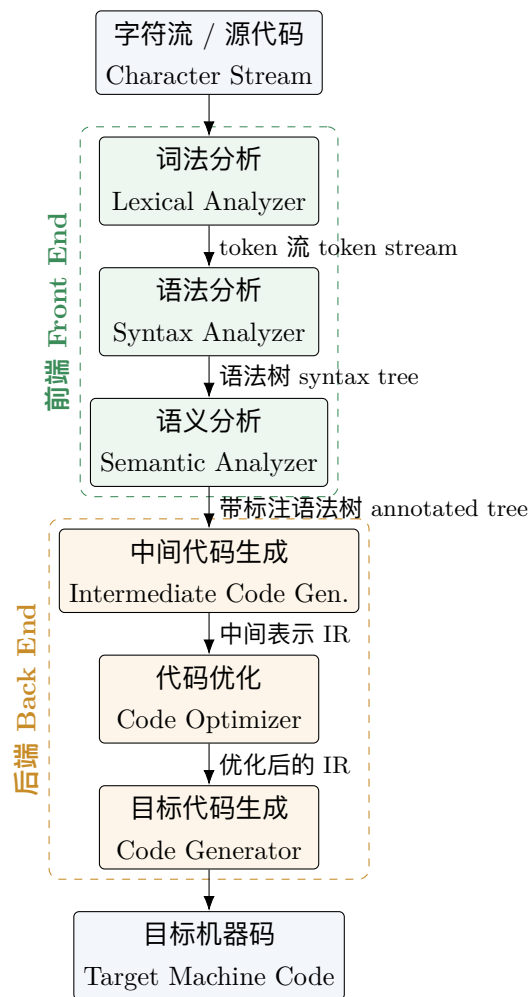
可重定位目标文件 (relocatable object, .o)：还没确定最终内存地址、需要与其他模块拼接的机器码。**链接 (Linking)** 就是把多个 .o 和库函数 (如 `printf`) 的代码”拼装”并定地址，最终得到能直接运行的 `a.out`。

2.5 为什么要学编译原理 [p18]

- **写出更高质量代码**：理解编译器有助写出对编译器友好、可被优化的代码（虽然多数日常编程并不直接用到）。
- **职业方向**：编译器研究员、游戏/优化编译器工程师等（多数岗位并不强制要求，但这是高门槛方向）。
- **掌握学科核心设计理念**：**抽象 (abstraction)**、**自顶向下/自底向上 (top-down / bottom-up)** 等思想贯穿计算机学科。
- **理论与实践深度结合**：编译原理是少数把”形式语言理论”直接落到”可运行系统”的课程。

3 编译器的总体结构 (第 19–20 页)

编译器是一条**流水线 (pipeline)**：每个阶段消费上一阶段的输出，产生下一阶段的输入。整体分**前端 (Front End)**与**后端 (Back End)**。



前端 (Front End, ”分析”部分) [p20]

面向源程序，负责识别结构、理解语义、报告错误。包含：词法分析、语法分析、语义分析。简记为 *Analysis* (分析)。

后端 (Back End, ”综合”部分) [p20]

综合前端的分析结果，生成与源程序语义等价的目标程序。包含：中间代码生成 (产出 IR, Intermediate Representation)、代码优化、目标代码生成 (产出 Target Machine Code)。简记为 *Synthesis* (综合)。

为什么要分前端/后端

分离的好处是**可复用**：同一个前端（如 C 语言前端）可以接不同后端（生成 x86 / ARM 代码）；反过来同一个后端也能服务多种语言的前端。中间的 **IR** 就是连接两者的”通用语”。

4 贯穿全课的运行示例

课件用同一段代码贯穿所有阶段，建议把它当作”主线案例”记住：

源代码 [p12,21]

```
while (y < z) {
    int x = a + b;
    y += x;
}
```

它会依次变成

1. token 序列 (词法)
2. 抽象语法树 AST (语法)
3. 带标注 AST + 符号表 (语义)
4. 三地址码 IR (中间代码)
5. 优化后的 IR (优化)
6. x86 汇编/机器码 (目标代码)

5 阶段一：词法分析 (第 21–22 页)

词法分析 (Lexical Analysis) [p21]

扫描源程序字符流，识别并切分出有词法意义的单词 / 符号 (token, 词法单元)。

输入：源程序 (字符流)；输出：token 序列。

每个 token 表示为 (类别, 属性值)，例如关键字、标识符、常量、运算符。

示例 (对应主线案例) [p21]: while (y<z){ int x=a+b; y+=x; } 被切成

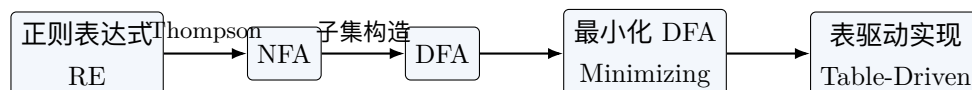
(keyword,while)(id,y)(sym,<)(id,z)(id,x)(id,a)(sym,+)(id,b)(sym,;)...

判据: token 是否合法? [p21]

词法分析还要判断单词是否符合词法规则。例如 0var、\$num 不是合法标识符 (标识符不能以数字开头、不能含非法字符)。

5.1 词法分析的知识地图 [p22]

这张图是后续两讲 (词法分析) 的术语全景，核心是一条自动化生成链：



模式 / 正则表达式 (Pattern / Regular Expression, RE)

用来描述一类单词长什么样 (如”标识符 = 字母开头的字母数字串”)。一个 token 类别由一个 RE 指定。 [p22]

词素 (Lexeme)

源代码中真正匹配某个模式的那一段字符 (如具体的变量名 y)。token 由词素抽象而来。 [p22]

NFA (Nondeterministic Finite Automaton, 非确定有限自动机)

一种识别 RE 的状态机，同一输入可有多条转移路径。由 RE 经 Thompson 算法构造。[p22]

DFA (Deterministic Finite Automaton, 确定有限自动机)

每个状态对每个输入只有唯一转移，便于直接编程。由 NFA 经子集构造算法 (Subset Construction) 得到。[p22]

DFA 最小化 (Minimizing)

合并等价状态，得到状态最少的 DFA，节省实现开销。[p22]

表驱动实现 (Table-Driven Implementation)

把 DFA 的转移关系存成一张表，用查表方式高效驱动词法分析器。[p22]

一句话串起来

语言规范 $\rightarrow$ 用 RE 描述每类 token $\rightarrow$ Thompson 转 NFA $\rightarrow$ 子集构造转 DFA $\rightarrow$ 最小化 $\rightarrow$ 表驱动，生成词法分析器 (Lexical Analyzer)。

6 阶段二：语法分析（第 23–25 页）

语法分析 (Syntax Analysis / Parsing) [p23]

解析 token 序列，构造出语法分析树 (syntax tree)，常用其精简形式抽象语法树 (AST, Abstract Syntax Tree)。

输入：token 流；输出：语法树。

同时判断输入程序是否符合语法规则（如 `x**`、`a += 5`；是否合法）。

词法 vs. 语法的分工

词法管“单词拼得对不对”（`0var` 非法）；语法管“句子结构对不对”（`x**` 是合法 token 但语法不通）。

6.1 基础概念 [p24]

乔姆斯基文法层次 (Chomsky's Grammar Hierarchy)

按表达能力把文法分四类：0 型无限制 (Unrestricted) $\supset$ 1 型上下文有关 (Context-Sensitive) $\supset$ 2 型上下文无关 (Context-Free, CFG) $\supset$ 3 型正则 (Regular)。编程语言的语法主要用 2 型 CFG 描述，词法用 3 型。[p24]

推导 (Derivation)

从文法的开始符号出发，反复用产生式替换，最终得到具体句子的过程（“文法 $\rightarrow$ 语言”）。[p24]

最左 / 最右推导 (Leftmost / Rightmost Derivation)

每步总是替换最左（或最右）的非终结符；与两类分析方法对应。[p24]

语法分析树 (Parse Tree)

推导过程的树形表示。[p24]

二义性 (Ambiguity)

同一句子存在两棵不同的语法树，会造成歧义；可通过规定优先级 (precedence) 与结合性 (associativity) 来消除。[p24]

6.2 两大类分析方法 [p25]

自顶向下分析 (Top-Down Parser) [p25]

从开始符号出发”往下猜”，尝试推出输入串。关键工具与概念：

First集 / Follow 集 (预测下一步用哪条产生式)、消除左递归 (Remove Left Recursion)、LL(1) 文法 / LL(1) 分析表、递归下降分析 (含回溯, RD-backtrack) 与预测分析 (Predictive Parser, 无回溯)。

自底向上分析 (Bottom-Up Parser) [p25]

从输入串出发”往上归约 (reduce)”到开始符号。主力是 LR 家族 (LR family)：

LR(0) (由项 item + 闭包 CLOSURE + GOTO 函数构造自动机与分析表，可能有冲突 conflicts)；
SLR(1) (用 Follow 集限制归约动作)； LR(1) (LR(0) 项 + 向前看符号 lookahead，最精确)；
LALR(1) (在 LR(1) 与 SLR(1) 间折中，工具常用)。

项 (Item)

带有”分析进度圆点”的产生式，记录已识别到哪。[p25]

闭包 (CLOSURE) 与 GOTO 函数 (GOTO Function)

构造 LR 自动机状态与状态间转移的两个基本运算。[p25]

冲突 (Conflict)

分析表某格出现多个动作 (移进/归约冲突等)，说明文法对该分析方法不够强。[p25]

向前看 (Lookahead)

多看后续若干符号来消除冲突，是 LR(1)/LALR(1) 比 LR(0) 强的原因。[p25]

记忆口诀

LL = 自顶向下、最左推导；LR = 自底向上、最右推导的逆。表达能力： $LR(1) \supset LALR(1) \supset SLR(1) \supset LR(0)$ 。语法分析是本课最重的部分（8 讲）。

7 阶段三：语义分析（第 26–27 页）

语义分析 (Semantic Analysis) [p26]

在语法结构基础上进一步分析含义。

输入：语法树；输出：带标注的语法树 (Annotated / Decorated AST) + 符号表 (Symbol Table)。

工作：收集标识符的属性信息（如类型 type、作用域 scope），并检查程序是否符合语义规则（如变量未声明就使用、重复声明）。

直观理解

语法树只知道”结构对”，但不知道”x 是 int 还是 bool、x 是否声明过”。语义分析就是给这棵树”贴上类型等标签”（所以叫带标注的 AST），同时用符号表登记每个名字的信息。

7.1 语义分析的知识地图 [p27]

语法制导定义 (SDD, Syntax-Directed Definitions)

在文法产生式上附加语义规则，给文法符号关联属性 (attributes)。[p27]

综合属性 (Synthesized Attribute)

由子节点”往上”计算得到的属性（自下而上）。[p27]

继承属性 (Inherited Attribute)

由父/兄节点”往下/横向”传来的属性。[p27]

依赖图 (Dependence Graph) 与拓扑序 (Topological Order)

属性间的计算依赖关系构成依赖图，按拓扑序求值才能保证先算被依赖者。[p27]

S-属性定义 (S-SDD)

只含综合属性，天然适合 LR/自底向上分析。[p27]

L-属性定义 (L-SDD)

综合 + 受限的继承属性，适合 LL/自顶向下分析。[p27]

语法制导翻译 (SDT, Syntax-Directed Translation)

把语义规则落成**语义动作 (semantic actions)**，嵌入产生式不同位置执行（如 LL 用 marker、LR 借助分析栈 stack）。[p27]

符号表 (Symbol Table)

登记标识符的类型、作用域、所属方法/类等；服务于**类型检查 (type checking)**与**作用域 (scope)**管理。[p27]

8 阶段四：中间代码生成（第 28–29 页）

中间代码生成 (Intermediate Code Generation) [p28]

做初步翻译，生成与源程序等价的**中间表示 (IR, Intermediate Representation)**。

输入：（带标注的）语法树；**输出：**IR。

作用：在源语言与目标语言之间架桥，让翻译更易实现，也利于某些优化算法。常见形式：**三地址码 (TAC, Three-Address Code)**。

主线案例的三地址码 [p28]：

```
                goto L1
L2:             t1 := a + b
                x  := t1
                t2 := y + x
                y  := t2
L1:             if y < z goto L2
```

为什么叫”三地址码”

每条指令最多含**三个地址 (操作数)**，形如 $x := y \text{ op } z$ ，结构简单、接近机器又独立于具体机器，非常适合做优化与后续翻译。t1, t2 是编译器引入的**临时变量 (temporary)**。

8.1 中间代码的知识地图 [p29]**抽象语法树 / DAG (AST / Directed Acyclic Graph)**

两种高层 IR；DAG 通过共享公共子表达式，比 AST 更紧凑。[p29]

三地址码 (Three-Address Code)

低层 IR，可用**四元式 (quadruples)**、**三元式 (triples)**、**间接三元式 (indirect triples)** 三种数据结构表示。[p29]

静态单赋值 (SSA, Static Single Assignment)

每个变量只被赋值一次的 IR 形式，极大方便优化。[p29]

布尔表达式翻译 (Boolean Expression)

可用直接翻译或基于跳转的间接（跳转 jumping）翻译处理控制流。[p29]

回填 (BackPatching)

一种一趟 (one-pass) 生成跳转代码的技术：先留空跳转目标，确定后再“填回去”，避免两趟扫描。[p29]

静态类型检查 (Static Type Check)

在生成 IR 阶段配合做类型检查。[p29]

9 阶段五：代码优化（第 30–31 页）

代码优化 (Code Optimization) [p30]

对中间代码进行加工变换，使其更优（代码更短、性能更高、内存更省）。

输入：IR；输出：优化后的 IR。

特点：本阶段优化机器无关 (machine independent)。例：公共子表达式识别、运算操作替换。

课堂 Quiz：这里做了什么优化？ [p30]

对比优化前后：原 IR 在循环体内每轮都算 $t1 := a+b$ ；优化后把 $t1 = a + b$ 提到循环外，循环内直接用 $t1$ 。这是循环不变量外提 (loop-invariant code motion) 配合公共子表达式消除的效果——因为 a 、 b 在循环中不变，无需重复计算。

9.1 代码优化的知识地图 [p31]

优化分两大类：与代码相关的变换 (Code-related) 和与布局相关的变换 (Layout-related)。

基本块 (Basic Block)

顺序执行、只能从开头进、从结尾出的最大指令序列；是优化的基本单位。[p31]

控制流图 (CFG, Control-Flow Graph)

以基本块为节点、跳转关系为边的图，描述程序执行路径。[p31]

局部优化 (Local Optimizations)

在单个基本块内做的优化。[p31]

全局优化 (Global Optimizations)

跨基本块的优化，需借助数据流分析。[p31]

数据流分析 (Dataflow Analysis)

沿 CFG 分析变量取值/活跃性等信息，为全局优化提供依据。[p31]

死代码消除 (Dead Code Elimination)

删除永远不会被用到、不影响结果的代码。 [p31]

公共子表达式消除 (Common Subexpression Elimination)

相同的计算只做一次，复用结果。 [p31]

常量传播 / 常量折叠 (Constant Propagation / Constant Folding)

把已知常量沿程序传播，并在编译期直接算出常量表达式（如 $3+4 \rightarrow 7$ ）。 [p31]

10 阶段六：目标代码生成（第 32–33 页）

目标代码生成 (Target Code Generation) [p32]

为特定机器产生目标代码（如汇编/机器码）。

输入：（优化后的）IR；输出：目标代码。

核心工作：**寄存器分配 (register allocation)** ——把频繁访问的数据放进寄存器；**指令选择 (instruction selection)** ——用机器指令实现 IR 操作；以及进一步的**机器相关优化**（如寄存器与访存优化）。

主线案例最终的 x86 片段 [p32]：

```
mov edx, DWORD PTR [rbp-12]
mov eax, DWORD PTR [rbp-16]
add eax, edx
mov DWORD PTR [rbp-20], eax
jmp L2
L3:mov eax, DWORD PTR [rbp-20]
add DWORD PTR [rbp-4], eax
L2:mov eax, DWORD PTR [rbp-4]
cmp eax, DWORD PTR [rbp-8]
jl L3
```

10.1 目标代码生成的知识地图 [p33]

指令选择 (Instruction Selection)

为每个 IR 操作挑选合适的机器指令。 [p33]

寄存器分配 (Register Allocation)

决定哪些变量驻留在有限的寄存器中（寄存器比内存快得多）。 [p33]

指令调度 / 排序 (Instruction Scheduling / Ordering)

重排指令顺序以提升流水线效率。 [p33]

活动记录 (Activation Record)

函数调用时在栈上为其分配的信息块（参数、局部变量、返回地址等）。 [p33]

调用者 / 被调用者约定 (Caller / Callee Conventions)

规定函数调用时谁负责保存哪些寄存器、如何传参。 [p33]

窥孔优化 (Peephole Optimization)

在一个小”窗口”内对相邻几条目标指令做局部改写，去除冗余。 [p33]

面向对象相关

还涉及方法分派 (Dispatch)、继承 (Inheritance) 等的目标代码实现。 [p33]

11 全流程一图速记

把主线案例 `while (y<z){int x=a+b; y+=x;}` 走完整条流水线：

表 3: 六阶段：输入 → 输出一览 (含主线案例形态)

阶段	输入	输出	案例中的形态
词法分析	字符流	token 序列	(keyword,while)(id,y)...
语法分析	token 序列	语法树 AST	while 子树 / 赋值子树
语义分析	AST	带标注 AST + 符号表	每个节点标上 int/bool 类型
中间代码	带标注 AST	中间表示 IR	三地址码 <code>t1:=a+b...</code>
代码优化	IR	优化后 IR	<code>a+b</code> 提出循环外
目标代码	优化后 IR	目标机器码	<code>x86 mov/add/cmp/jl...</code>

考点速记

- 前端三件套：词法、语法、语义 (分析 Analysis)；后端三件套：中间代码、优化、目标代码 (综合 Synthesis)。
- 各阶段产物链：字符流 → token → AST → 带标注 AST+ 符号表 → IR → 优化 IR → 目标码。
- 编译 / 解释 / JIT：AOT 性能最好、解释最灵活、JIT 折中；Java = 字节码 + 解释/JIT。
- C 编译四步：预处理.i → 编译.s → 汇编.o → 链接 a.out。
- 词法链：RE $\xrightarrow{\text{Thompson}}$ NFA $\xrightarrow{\text{子集构造}}$ DFA → 最小化 → 表驱动。
- 语法两派：LL (自顶向下/最左) vs. LR (自底向上/最右逆)；LR(1) 最强、LALR(1) 常用。